Comparative Analysis of YOLOv8n Finetuning Strategies

*Evaluating model adaptation techniques for accurate game card recognition across varied training setups

1st Kremmer, Moritz
IT-Universitetet i København
Copenhagen, Denmark
moritzkremmer@icloud.com

2nd Seibel, Alexander
IT-Universitetet i København
Copenhagen, Denmark
alexanderse1501@gmail.com

3rd Stärk, Johannes
IT-Universitetet i København
Copenhagen, Denmark
johannes.joel.staerk@gmail.com

I. Introduction

Detecting small and visually similar objects is a challenging problem in computer vision and appears in many industrial applications, such as identifying tiny surface defects in manufacturing or detecting small printed symbols in logistics systems. Recognizing the rank and suit symbols printed on playing cards presents a similar challenge: the symbols are extremely small, highly similar across classes, and vary in orientation. These characteristics align well with the broader difficulties described in prior works regarding fine-grained visual categorization, where subtle inter-class differences, pose sensitivity, and limited pixel information make discrimination difficult [1]. These difficulties are further amplified when objects occupy only a small region of the image, limiting the amount of visual information available for discrimination. As a result, recognizing the small card symbols requires a model capable of extracting fine-grained local cues despite limited resolution and dataset size.

To address this, transfer learning is a common approach. Models pre-trained on large datasets such as COCO provide general visual features that can be adapted to new tasks with comparatively little data [2]. This adaptation is commonly achieved by fine-tuning part or all of the pre-trained network. The degree of fine-tuning influences both performance and training stability, as highlighted in transfer learning surveys [3]. A known drawback of fine-tuning is catastrophic forgetting, where performance on the original pre-training task may degrade as the model adapts to a new domain [4]. This issue is particularly relevant in our setting, since fine-tuning a pretrained model on a small, specialised dataset increases the likelihood that updates overwrite general-purpose features.

Given these constraints, the choice of detector is essential. The YOLO family of one-stage detectors is known for fast inference and effective multi-scale feature extraction [5], [6]. These properties make it a strong candidate for detecting the very small rank and suit symbols on playing cards. In contrast, two-stage detectors such as Faster R-CNN rely on a more complex pipeline and operate on relatively coarse feature maps, which can reduce performance on very small objects [7]. A recent review further highlights these differences, reporting that YOLOv8 achieves higher accuracy and significantly faster inference than Faster R-CNN across several benchmarks, making it better suited for efficient and real-time detection scenarios [8]. Beyond two-stage architectures, transformer-based detectors such as DETR require extremely long training schedules, large datasets, and are known to struggle particularly with small objects due to their coarse-to-fine attention mechanism [9].

In addition, a closely related study fine-tunes YOLOv8 for a fine-grained fruit detection task and shows that partially unfreezing the backbone can substantially improve accuracy without causing severe catastrophic forgetting [10]. Their findings motivate our investigation but differ from our setting, where objects are significantly smaller, visually more similar, and appear in more constrained spatial layouts.

Based on these considerations, we fine-tune YOLOv8 under different degrees of layer freezing to assess how well the model adapts to this fine-grained, small-object task and to analyse the resulting error patterns. In particular, we formulate the following hypotheses:

- **H1:** Increasing the number of trainable layers during fine-tuning improves detection performance without causing catastrophic forgetting, as YOLOv8 retains useful pre-trained features while adapting to the fine-grained card-symbol domain.
- **H2:** Misclassifications predominantly occur between visually similar card symbols, reflecting the difficulty of discriminating fine-grained classes that occupy only a small fraction of the image.

II. Methods

The YOLOv8 models are provided by Ultralytics, an open-source organization that maintains the YOLO family of real-time object detectors [11]. YOLOv8 is a widely adopted one-stage, anchor-free detector. Being one-stage means that object localization and classification are performed in a single forward pass of the network. Unlike two-stage detectors (e.g. Faster R-CNN), which generate region proposals before classification, one-stage models directly predict bounding boxes and

class probabilities at each spatial location, resulting in lower computational overhead and latency. YOLOv8 is also anchor-free, meaning that bounding boxes are predicted directly from feature map points rather than from predefined anchor shapes, as in earlier YOLO versions. Anchor-based approaches require careful tuning of anchor sizes and aspect ratios, which can be suboptimal for small or variably sized objects [6].
Due to limited computational resources, we use the YOLOv8n variant, the smallest model.

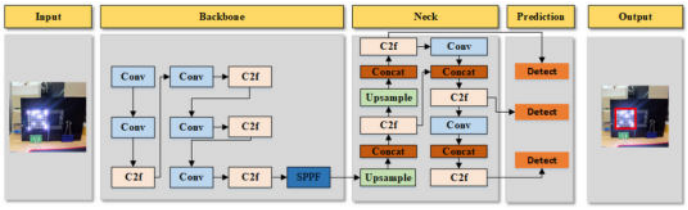

Fig. 1: Overview of YOLOv8 architecture: backbone, neck and head [12].

It follows a backbone–neck–head architecture (see Figure 1). The backbone processes the input image and produces a set of feature maps at different spatial resolutions, capturing information at multiple levels of abstraction. These feature maps are passed to the neck, which combines them using a feature pyramid structure to produce a set of multi-scale, semantically enriched feature representations [8], [13], [14]. The detection head operates on the multi-scale feature maps produced by the neck and is organized into two parallel branches: a classification branch and a regression branch. Each branch is optimized using a dedicated loss function, as illustrated in Figure 2, and the total training loss is obtained by summing these components [15].

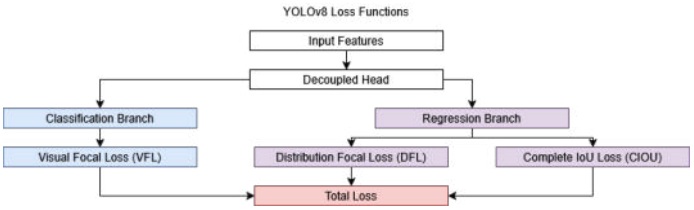

Fig. 2: Overview of YOLOv8 loss components: decoupled head with classification and regression branches [15].

A. Visual Focal Loss (VFL) – Classification

Focal Loss was introduced to handle class imbalance by down-weighting easy negatives and emphasizing hard examples [16]. YOLOv8 uses an asymmetric classification loss that treats positive and negative samples differently, allowing the model to emphasize high-quality predictions and reduce the impact of easy background examples [15]. This idea extends focal-loss concepts and is commonly referred to in the community as Visual Focal Loss (VFL), which is not an official Ultralytics term [17]:

- **Positive samples** ($q > 0$)**:** For positive samples, the target value q equals the IoU between prediction and ground truth. High-IoU positives receive higher weight:

$$L_{\text{pos}} = -q \cdot \log(p) \tag{1}$$

- **Negative samples** ($q = 0$)**:** Negative samples use a focal-weighted term to suppress easy background predictions, where α controls the overall weight and γ sharpens the focus on hard negatives:

$$L_{\text{neg}} = -\alpha(1-p)^{\gamma} \log(p) \tag{2}$$

B. Distribution Focal Loss (DFL)

Instead of predicting each box edge as a single value, the model outputs a distribution over discrete bins, allowing for sub-pixel accuracy and smoother gradients [18]–[20].
Originally this concept was introduced in a 2020 paper [21].
Given:

- y = ground-truth coordinate
- y_i and y_{i+1} = the two nearest bin centers
- S_i and S_{i+1} = predicted probabilities for these bins

The DFL loss is defined as:

$$L_{\text{DFL}} = -\left[(y_{i+1} - y)\log(S_i) + (y - y_i)\log(S_{i+1})\right] \tag{3}$$

This encourages the model to place probability mass near the true location, making it especially effective for detecting small or fine-grained objects. By modeling coordinates as distributions, DFL enables the network to learn more precise and stable box boundaries [18]–[20].

C. Complete IoU Loss (CIoU)

For bounding-box regression, YOLOv8 uses Complete IoU Loss (CIoU), introduced in 2020 [22], which incorporates additional geometric factors beyond simple overlap.
Traditional IoU loss only measures the overlap area between predicted and ground-truth boxes. This leads to two major problems [22]–[24]:

- No gradient when boxes do not overlap: the model cannot learn to move the box toward the target
- Ignoring geometry: boxes may have good overlap but incorrect center alignment or aspect ratio, resulting in unstable or slow convergence

CIoU addresses these issues by combining three geometric factors [22]–[24]:

1) Overlap (same as IoU)
2) Normalized center distance between boxes
3) Aspect ratio consistency

By considering all three factors together, CIoU provides stronger and more stable optimization signals. This helps the model converge to bounding boxes that are not only overlapping but also correctly positioned and shaped [23].
For our dataset of small card symbols, this is particularly beneficial. Precise center alignment and aspect-ratio stability are essential for accurate detection.

D. Relevance to Our Task

The combination of VFL, DFL, and CIoU is particularly well-suited for detecting card symbols in our dataset. Each component addresses a specific challenge:

- **Background suppression:** VFL down-weights easy background examples, letting the model focus on high-quality positives for mAP
- **Small objects:** DFL improves edge precision, which is critical for accurately localizing small symbols
- **Consistent shapes:** CIoU encourages well-aligned and tightly fitted bounding boxes, improving geometric accuracy

III. Experiments

A. Dataset characteristics

Our experiments utilize a playing-cards detection dataset containing images of standard 53-card decks (including Jokers) in various real-world scenes. The cleaned dataset consists of 1,855 images for training (84.0%), 231 for validation (10.5%), and 123 for testing (5.6%). The class distribution is moderately balanced: the Joker appears most often (2,388 instances), while all remaining card classes occur between roughly 750 and 1,800 instances (Figure 7).

On average, images contain 28.4 annotated objects (standard deviation 32.5, range 1–188; Figure 8), indicating high variability in object density and resulting in roughly 60,000 bounding boxes in total. Bounding-box statistics (Figures 9, 10, 11) show that most objects have normalized widths and heights below 0.1 and cover less than 2% of the image area, confirming that the dataset mainly comprises small objects. The aspect-ratio distribution has a strong peak for tall, card-like shapes (width/height < 1) and a secondary mode for wider boxes, reflecting rotated or partially visible cards. A heatmap of normalized bounding-box centers (Figure 12) shows that objects cover most of the image area with a slight concentration near the center. We characterize appearance variation using per-image mean intensity and pixel standard deviation (brightness and contrast). The histograms (Figure 13) show moderate variation without extreme low-light or overexposed cases. Overall, the dataset represents a high-density small-object detection setting with moderate variation in layout and illumination.

B. Model variants and freezing strategies

As described above, we adopt the YOLOv8n architecture as our base detector. For all experiments we start from the official pretrained weights and only vary which parts of the network are fine-tuned on the playing-card dataset, keeping all other hyperparameters fixed. This allows us to study how the depth of fine-tuning trades off performance and computational cost. We consider three configurations with increasing numbers of trainable parameters:

- **Head Only**: The backbone and feature-fusion neck are frozen; only the decoupled detection head is updated during training (0.76M trainable parameters - Figure 14)
- **Neck + Head**: The backbone is frozen, while both the neck and detection head are trainable (1.75M trainable parameters - Figure 15).
- **Entire Model**: All components, backbone, neck, and detection head, are trained end-to-end (3.02M trainable parameters - Figure 16).

C. Evaluation protocol

All models are evaluated on the held-out test split without test-time augmentation, using the standard YOLOv8 evaluation pipeline. We report overall precision, recall, mean Average Precision at IoU 0.5 (mAP50), mean Average Precision averaged over IoU thresholds 0.5–0.95 (mAP50–95), per-class AP, and confusion matrices.

A detection is counted as correct if its bounding box has Intersection-over-Union (IoU) ≥ 0.5 with a ground-truth box and the predicted class matches the ground-truth class. mAP50 is computed at this IoU threshold, while mAP50–95 averages AP over IoU thresholds from 0.5 to 0.95 in steps of 0.05. Precision and recall are computed using the same IoU matching rule.

During training, we also log training and validation losses (box, classification, DFL) and validation precision, recall, and mAP over epochs, which we later use to analyze training dynamics.

D. Overall test-set performance

Table I summarizes the test-set performance of the three fine-tuning strategies. Full end-to-end fine-tuning (*Entire Model*) achieves the best results with precision 0.82, recall 0.77, mAP50 0.85, and mAP50–95 0.52. Freezing the backbone but training the Neck + Head (*Neck + Head*) roughly halves the number of trainable parameters, but reduces mAP50–95 to 0.30. Updating only the detection head (*Head Only*) is parameter-efficient (about 25% of the trainable parameters of the Entire Model), but performance collapses to mAP50–95 of 0.08 with low precision and recall. Overall, the results indicate that adapting at least the neck in addition to the head is necessary to retain strong detection performance on this task.

TABLE I: Test Set Performance

Model	Precision	Recall	mAP@50	mAP@50-95
Entire Model	[0.82]	[0.77]	[0.85]	[0.52]
Neck + Head	[0.51]	[0.55]	[0.53]	[0.30]
Head Only	[0.18]	[0.28]	[0.16]	[0.08]

E. Training and validation dynamics

Figure 3 shows the evolution of total training and validation loss (sum of box, classification, and DFL losses) over 100 epochs for all three fine-tuning strategies. All configurations exhibit rapid loss reduction in the first 20–30 epochs followed by a gradual plateau, indicating stable convergence without signs of divergence. The *Entire Model* consistently attains the lowest losses, *Neck + Head* converges to intermediate values, and *Head Only* remains clearly higher throughout training. The small but persistent gap between training and validation curves suggests mild overfitting but no severe mismatch. Component-wise box, classification, and DFL loss curves for each configuration are provided in Figures 17, 18, 19 and they follow the same ordering.

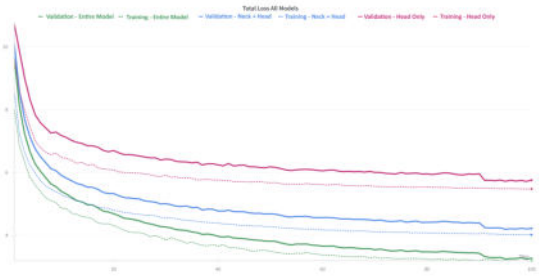

Fig. 3: Total training (solid) and validation (dashed) loss over 100 epochs for all three fine-tuning strategies

F. Per-class performance

To further characterize model behavior, class-wise confidence curves are examined for the fully-finetuned model. Across the 53 symbol classes, precision ranges from 0.59 to 0.97, recall from 0.47 to 0.98, mAP50 from 0.62 to 0.99, mAP50–95 from 0.34 to 0.68, and F1-score from 0.55 to 0.97. Thus all cards are detectable, but some symbols are substantially easier than others. Table II summarizes the three best and three worst classes ranked by mAP50–95. The best-performing symbols are dominated by low ranks: the 3 of Clubs (k3) is almost perfectly detected, followed by the 3 of Hearts (h3) and the 7 of Diamonds (r7). At the other end of the spectrum, the weakest classes are the 6 of Clubs (k6), 6 of Spades (s6), and King of Hearts (hh), which all show noticeably lower recall and mAP50–95.

Grouping by rank reveals a clearer pattern: rank–3 cards are easiest on average (mean mAP50–95 ≈ 0.63), followed by 10s, aces, and 7s, while 6s are the hardest group (mean mAP50–95 ≈ 0.46), with kings and 8s also slightly below the overall average.

Averaging over suits yields very similar per-suit mAP50–95 values, indicating no strong performance differences between hearts, clubs, diamonds, and spades.

G. Confidence behavior

For the Entire Model we also inspect per-class confidence curves. At low confidence thresholds (e.g. ≈ 0.1), the model retains almost all predicted bounding boxes, including many low-confidence detections. As a result, most ground-truth objects are matched by at least one prediction, leading to very few false negatives and recall values close to 1.0 for most classes (Figure 20). However, many incorrect detections are also included, which results in low precision in this regime (Figure 21).

As the confidence threshold increases, low-quality predictions are progressively filtered out. This causes precision to rise rapidly, reaching values around 0.8–0.9 for most classes once the threshold exceeds approximately 0.3. At the same time, recall begins to decrease, as some true objects are no longer detected when their associated predictions fall below the threshold.

The opposing trends of precision and recall are summarized by the F1–confidence curves (Figure 4). For most classes, F1 reaches a clear maximum between confidence values of approximately 0.3 and 0.5, indicating the range in which the balance between false positives and false negatives is optimal. Increasing the threshold beyond about 0.6 yields only marginal gains in precision, while recall drops sharply, causing a noticeable decline in F1.

Overall, these curves indicate that operating the model at a confidence threshold in the range of 0.3–0.4 provides a stable trade-off between precision and recall across classes.

TABLE II: Best and worst classes for the Entire Model (validation set), ranked by mAP50–95.

Class	Precision	Recall	mAP50	mAP50–95
Best classes				
k3 (3 of Clubs)	0.97	0.98	0.99	0.68
h3 (3 of Hearts)	0.89	0.94	0.96	0.63
r7 (7 of Diamonds)	0.86	0.93	0.96	0.61
Worst classes				
k6 (6 of Clubs)	0.59	0.58	0.63	0.34
s6 (6 of Spades)	0.66	0.47	0.62	0.37
hh (King of Hearts)	0.73	0.60	0.69	0.39

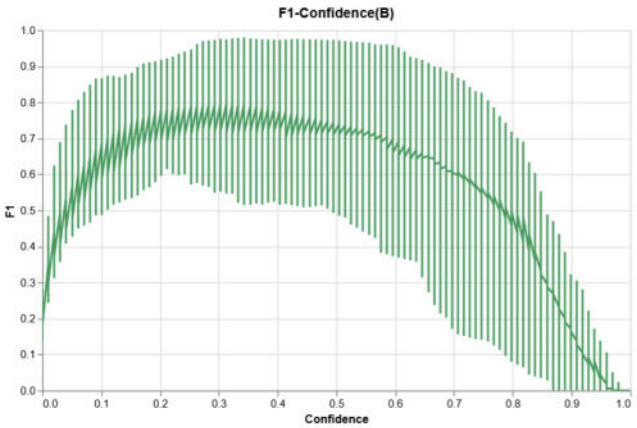

Fig. 4: F1-score across different confidence thresholds, showing class-level spread.

H. Qualitative Analysis

The qualitative results of example predictions from the models (Figures 5, 22, 23, 24) demonstrate substantial differences in detection quality across the three fine-tuning approaches. The Entire Model configuration produces dense, comprehensive detections with numerous bounding boxes covering individual playing cards across all frames, though this leads to occasional overlapping predictions and redundant detections on the same card. The Head Only configuration generates significantly sparser detections with fewer bounding boxes per frame, resulting in many missed cards (false negatives) and gaps in coverage, particularly in frames with multiple cards or complex arrangements. The Neck + Head approach achieves an intermediate balance, detecting more cards than the Head Only model while maintaining better precision than the Entire Model by reducing excessive overlapping predictions. Notable differences include the Entire Model's superior ability to detect cards in various orientations and positions throughout the entire frame, while the Head Only model struggles particularly with challenging scenarios such as partially visible cards, cards at the edges of frames, and scenes with multiple overlapping cards where disambiguation is required. The Neck + Head configuration shows moderate performance in these difficult cases, successfully detecting prominent cards while occasionally missing those with partial occlusion or unconventional positioning.

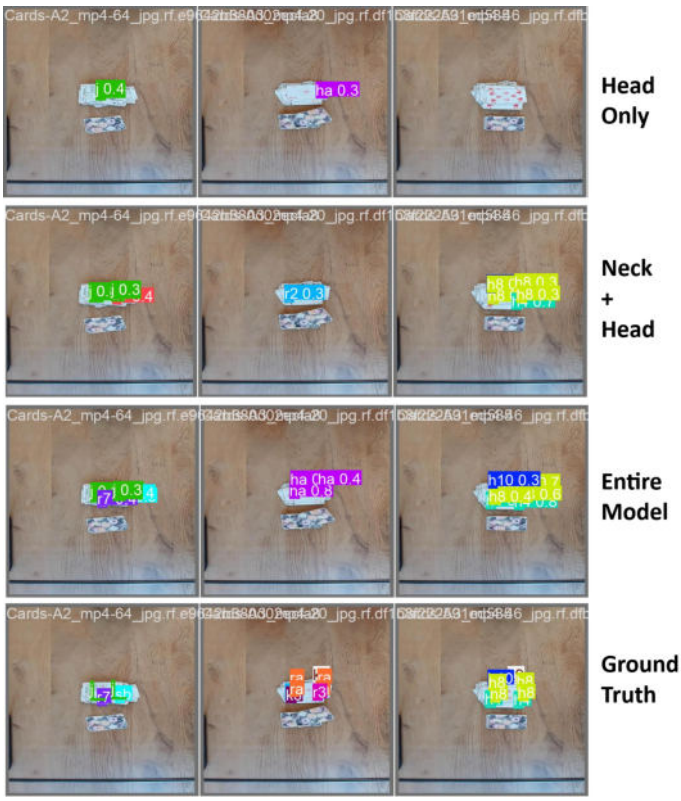

Fig. 5: Comparison of Detection Examples from all finetuning stages

IV. Discussion

We focus our discussion on two hypotheses that address central aspects of the model's behavior: trade-off between model adaptability and catastrophic forgetting during fine-tuning and the error patterns arising from visual similarities between card symbols. We analyze each hypothesis below.

A. Hypothesis: Higher model capacity leads to better performance and not catastrophic forgetting

The results across the three model variants strongly support the hypothesis that a higher model capacity, or more trainable layers, leads to better performance without causing catastrophic forgetting. As illustrated in Figure 25 in the Appendix the total number of misclassifications consistently decreases from the Head Model to the Neck + Head Model to the Entire Model. This trend is reflected in all evaluation metrics. The Entire Model achieves the highest mean precision (0.82), recall (0.77), mAP50 (0.85), and mAP50-95 (0.52), followed by the Neck + Head and Head Models.

These results suggest that restricting the number of layers hinders the model's ability to adapt its feature representations to the fine-grained structure of the card symbols. The precision-recall plots (Figures 26-28) highlight this. The Head Model clusters around low precision and recall. The Neck + Head Model shows inconsistent performance across a broader range. The Entire Model features many high-performing classes. These results suggest that successfully recognizing small, visually similar symbols requires adjustments in not only the detection head, but also in mid-level and early backbone features.

Importantly, increasing the model's capacity did not result in catastrophic forgetting, which would have degraded performance. One possible explanation is that the target domain, printed card symbols, is visually simple and similar enough to the pretrained data that existing features can be refined rather than overwritten. Allowing more layers to update helps the model specialize; heavily frozen variants, on the other hand, underperform because they cannot adapt sufficiently to the task.

B. Hypothesis: Visually Similar Cards Are Misclassified More Often

The results from all three model variants support the hypothesis that visually similar cards are misclassified more often. Figure 6, which shows the confusion matrix of the fully trainable model, clearly illustrates this: the most frequent errors occur between symbol pairs with similar shapes. Examples include 6 and 9 (rotational ambiguity) and 5 and 8. There is also confusion between the same number in different suits, such as the Ace of Hearts versus the Ace of Diamonds, where subtle stylistic or stroke-width differences are not distinctive enough for the model. These examples demonstrate how small visual variations can be difficult to discern when the symbols occupy only a small area of the image.

This observation aligns with the model's overall detection metrics. While the Entire Model achieves a strong mean average

precision (mAP) at 50% (mAP50) of 0.85 and a respectable mAP at 95% (mAP95) of 0.53, the drop at higher intersection over union (IoU) thresholds reflects the model's difficulty in precisely distinguishing fine-grained symbol shapes. For visually similar symbols, such as 6/8/9 clusters or ace variants across suits, the model often detects the correct region but assigns the wrong class. This emphasizes that classification, not localization, is the key bottleneck.

The confusion matrices (Figures 29, 30) further confirm that these misclassification patterns persist across all architectures despite differences in the number of fine-tuned layers. Rotationally symmetric pairs, such as 6 vs. 9; numerically similar shapes, such as 5 vs. 8; and stylistically close suit variants frequently appear among the top confusions in every model. This consistency suggests that the challenge stems from inherent symbol similarity and YOLOv8's limited ability to distinguish closely overlapping feature clusters.

These findings point to several promising directions for improvement. Higher-resolution inputs or symbol-centered cropping could preserve the discriminative detail necessary to reliably distinguish between similar shapes, such as 5 and 8. Rotation-aware architectures or auxiliary orientation prediction tasks could reduce ambiguity in pairs such as 6 and 9.

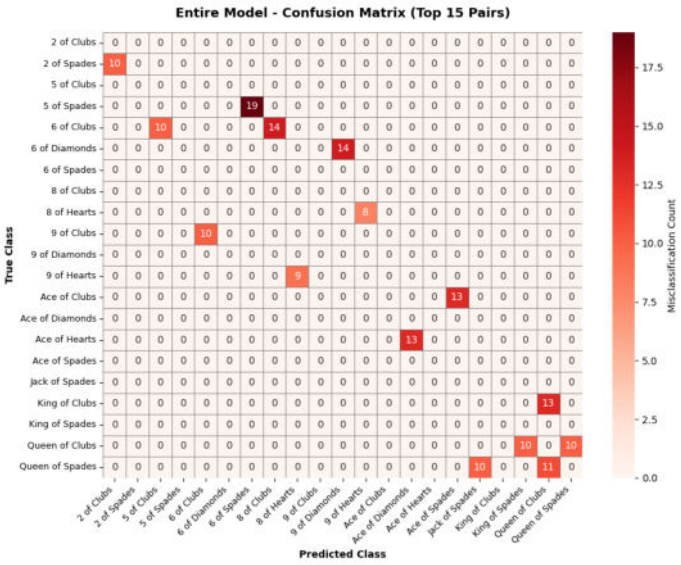

Fig. 6: Misclassifications Confusion Matrix for Entire Model

V. Conclusions

This study compared three YOLOv8n fine-tuning strategies for detecting small, visually similar playing card symbols: training the Entire Model, the Neck + Head, and the Head Only. The Entire Model achieved the best performance with mAP50 of 0.85, mean precision of 0.82, and mean recall of 0.77, followed by the Neck + Head and head-only configurations. No evidence of catastrophic forgetting was observed when fine-tuning all layers. The main source of errors across all variants was misclassification between visually similar symbols, particularly rotationally ambiguous pairs (6 vs. 9) and suit variants of the same rank (e.g., different aces). These findings demonstrate that fine-grained small-object detection benefits from maximizing trainable model capacity when domain shift is modest, while classification accuracy on subtle visual features remains the primary bottleneck.

REFERENCES

[1] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, "The Caltech-UCSD Birds-200-2011 Dataset," California Institute of Technology, Technical Report CNS-TR-2011-001, 2011. [Online]. Available: https://authors.library.caltech.edu/records/cvm3y-5hh21

[2] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in *Computer Vision – ECCV 2014*, D. Fleet, T. Pajdla, B. Schiele, and T. Tuytelaars, Eds. Cham: Springer International Publishing, 2014, vol. 8693, pp. 740–755, series Title: Lecture Notes in Computer Science. [Online]. Available: http://link.springer.com/10.1007/978-3-319-10602-1_48

[3] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He, "A Comprehensive Survey on Transfer Learning," *Proceedings of the IEEE*, vol. 109, no. 1, pp. 43–76, Jan. 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9134370/

[4] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, Mar. 2017. [Online]. Available: https://pnas.org/doi/full/10.1073/pnas.1611835114

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2015, version Number: 5. [Online]. Available: https://arxiv.org/abs/1506.02640

[6] Ultralytics, "YOLOv8 vs YOLOv5: Evolution of Real-Time Object Detection." [Online]. Available: https://docs.ultralytics.com/compare/yolov8-vs-yolov5/

[7] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," 2015, version Number: 3. [Online]. Available: https://arxiv.org/abs/1506.01497

[8] M. Sohan, T. Sai Ram, and C. V. Rami Reddy, "A Review on YOLOv8 and Its Advancements," in *Data Intelligence and Cognitive Informatics*, I. J. Jacob, S. Piramuthu, and P. Falkowski-Gilski, Eds. Singapore: Springer Nature Singapore, 2024, pp. 529–545, series Title: Algorithms for Intelligent Systems. [Online]. Available: https://link.springer.com/10.1007/978-981-99-7962-2_39

[9] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-End Object Detection with Transformers," 2020, version Number: 3. [Online]. Available: https://arxiv.org/abs/2005.12872

[10] V. Gandhi and S. Gandhi, "Fine-Tuning Without Forgetting: Adaptation of YOLOv8 Preserves COCO Performance," 2025, version Number: 1. [Online]. Available: https://arxiv.org/abs/2505.01016

[11] Ultralytics, "Revolutionizing the World of Vision AI." [Online]. Available: https://www.ultralytics.com

[12] H. Herfandi, O. S. Sitanggang, M. R. A. Nasution, H. Nguyen, and Y. M. Jang, "Real-Time Patient Indoor Health Monitoring and Location Tracking with Optical Camera Communications on the Internet of Medical Things," *Applied Sciences*, vol. 14, no. 3, p. 1153, Jan. 2024. [Online]. Available: https://www.mdpi.com/2076-3417/14/3/1153

[13] Ultralytics, "Entdecken Sie Ultralytics YOLOv8." [Online]. Available: https://docs.ultralytics.com/de/models/yolov8/

[14] M. Yaseen, "What is YOLOv8: An In-Depth Exploration of the Internal Features of the Next-Generation Object Detector," 2024, version Number: 1. [Online]. Available: https://arxiv.org/abs/2408.15857

[15] DataXujing, "Loss Functions YOLOv8," Apr. 2025. [Online]. Available: https://deepwiki.com/DataXujing/YOLOv8/2.1-loss-functions

[16] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," 2017, version Number: 2. [Online]. Available: https://arxiv.org/abs/1708.02002

[17] Ultralytics, "Reference for ultralytics/utils/loss.py." [Online]. Available: https://docs.ultralytics.com/reference/utils/loss/

[18] E. Kim, "Distribution Focal Loss (DFL) in YOLO colab," Jun. 2025. [Online]. Available: https://medium.com/@elvenkim1/dual-focal-loss-dfl-in-yolo-colab-0c9ac722f917

[19] bubbliiiing, "Distribution Focal Loss (DFL)," Apr. 2025. [Online]. Available: https://deepwiki.com/bubbliiiing/yolov8-pytorch/2.4-distribution-focal-loss-(dfl)

[20] J. Torres, "What is DFL loss in yolov8?" Oct. 2024. [Online]. Available: https://yolov8.org/what-is-dfl-loss-in-yolov8/

[21] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, "Generalized Focal Loss: Learning Qualified and Distributed Bounding Boxes for Dense Object Detection," 2020, version Number: 1. [Online]. Available: https://arxiv.org/abs/2006.04388

[22] Z. Zheng, P. Wang, D. Ren, W. Liu, R. Ye, Q. Hu, and W. Zuo, "Enhancing Geometric Factors in Model Learning and Inference for Object Detection and Instance Segmentation," 2020, version Number: 4. [Online]. Available: https://arxiv.org/abs/2005.03572

[23] P. Chandra, "IoU Loss Functions for Faster & More Accurate Object Detection," Jun. 2023. [Online]. Available: https://learnopencv.com/iou-loss-functions-object-detection/

[24] mausam, "IoU,GIoU,DIoU,CIoU Loss," Mar. 2024. [Online]. Available: https://datachemy.substack.com/p/iougioudiouciou-loss

APPENDIX

Label Dictionary

TABLE III: Translation of German/Dutch Card Codes to English

Original	English Name
h2	2 of Hearts
h3	3 of Hearts
h4	4 of Hearts
h5	5 of Hearts
h6	6 of Hearts
h7	7 of Hearts
h8	8 of Hearts
h9	9 of Hearts
h10	10 of Hearts
ha	Ace of Hearts
hb	Jack of Hearts
hh	King of Hearts
hv	Queen of Hearts
k2	2 of Clubs
k3	3 of Clubs
k4	4 of Clubs
k5	5 of Clubs
k6	6 of Clubs
k7	7 of Clubs
k8	8 of Clubs
k9	9 of Clubs
k10	10 of Clubs
ka	Ace of Clubs
kb	Jack of Clubs
kh	King of Clubs
kv	Queen of Clubs
r2	2 of Diamonds
r3	3 of Diamonds
r4	4 of Diamonds
r5	5 of Diamonds
r6	6 of Diamonds
r7	7 of Diamonds
r8	8 of Diamonds
r9	9 of Diamonds
r10	10 of Diamonds
ra	Ace of Diamonds
rb	Jack of Diamonds
rh	King of Diamonds
rv	Queen of Diamonds
s2	2 of Spades
s3	3 of Spades
s4	4 of Spades
s5	5 of Spades
s6	6 of Spades
s7	7 of Spades
s8	8 of Spades
s9	9 of Spades
s10	10 of Spades
sa	Ace of Spades
sb	Jack of Spades
sh	King of Spades
sv	Queen of Spades
j	Joker
pile-face-down	Pile of cards face down
pile-face-up	Pile of cards face up

III.A EDA

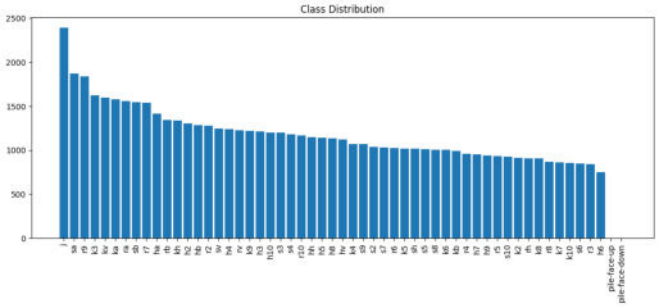

Fig. 7: Class distribution across 53 card classes

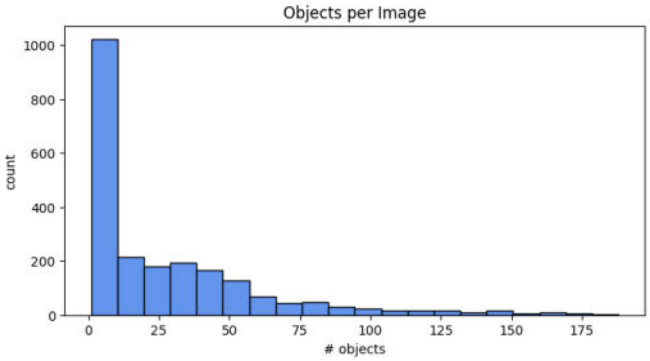

Fig. 8: Objects per image

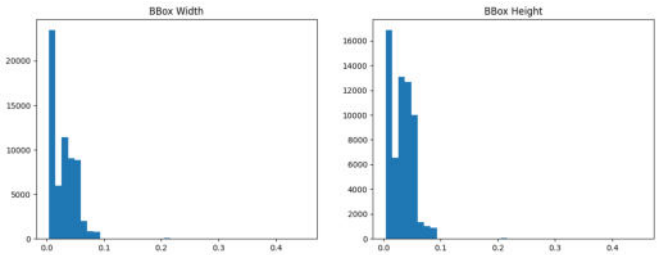

Fig. 9: Bounding-box width vs. height

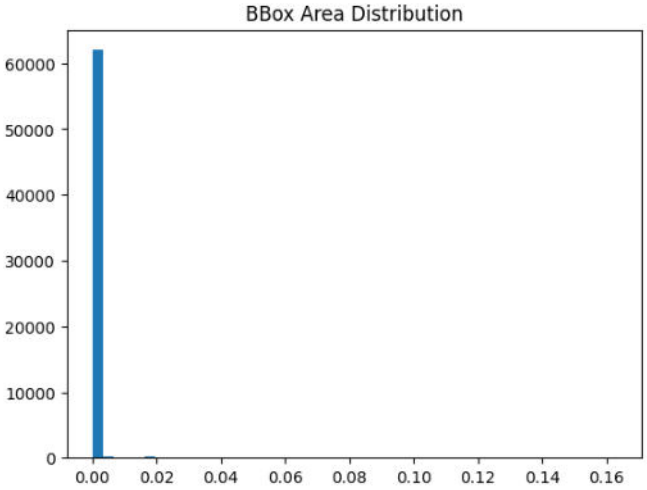

Fig. 10: Bounding-box area distribution

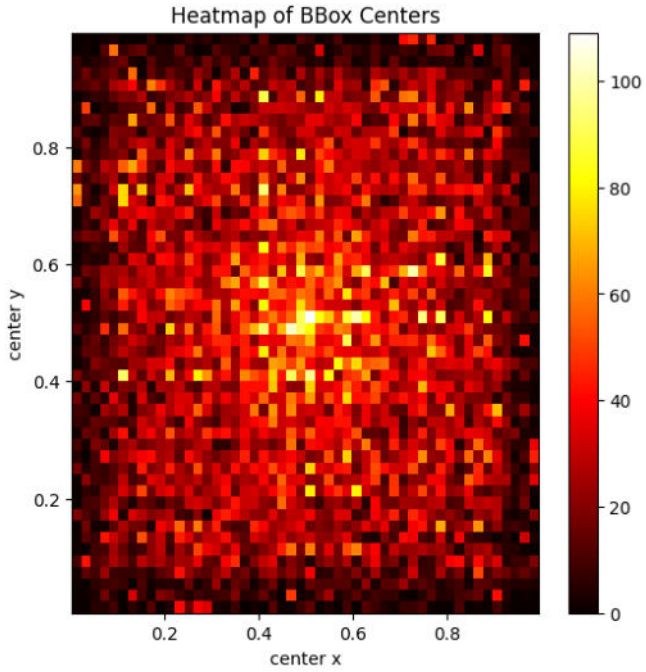

Fig. 12: Bounding-box center heatmap

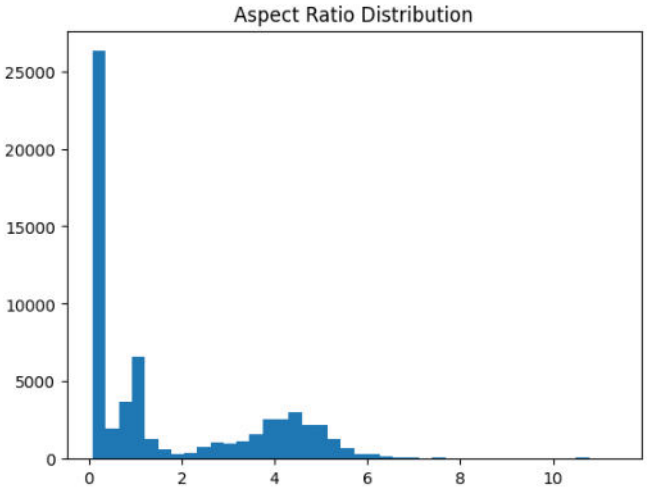

Fig. 11: Bounding-box aspect ratio

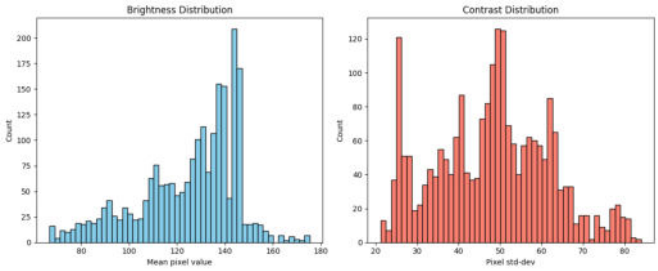

Fig. 13: Brightness and contrast statistics

III.B Model variants and freezing strategies

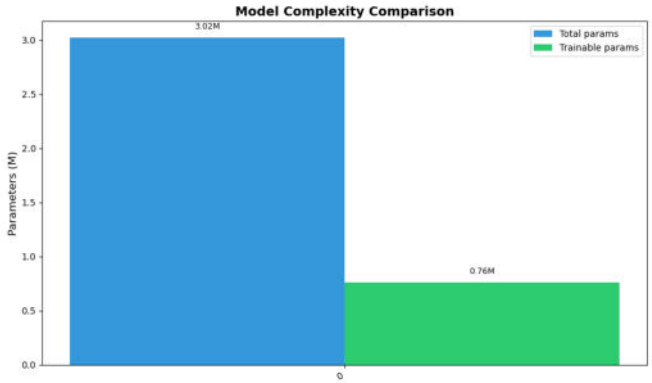

Fig. 14: Head-only model parameters

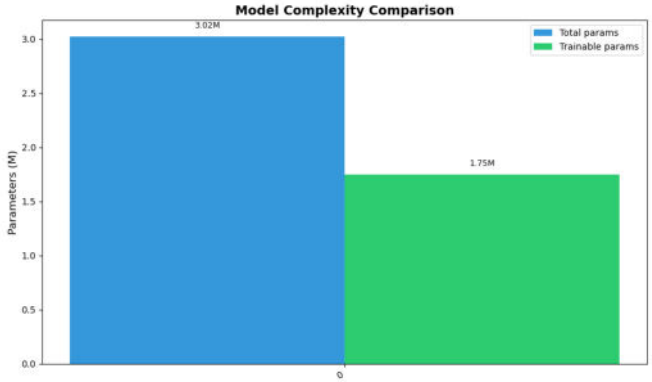

Fig. 15: Neck + Head model parameters

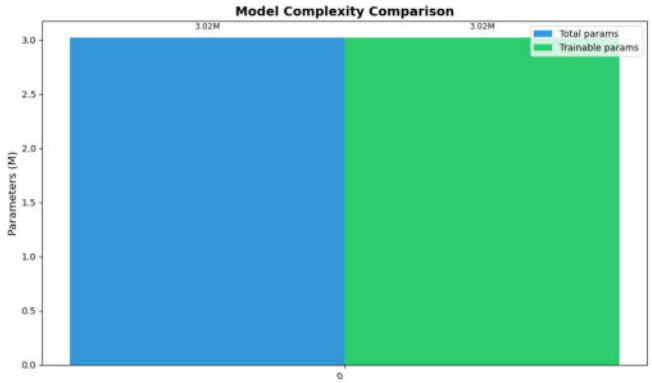

Fig. 16: Entire model parameters

III.E Training and validation dynamics

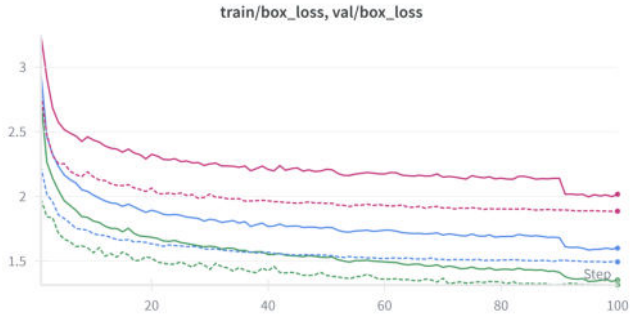

Fig. 17: Box loss

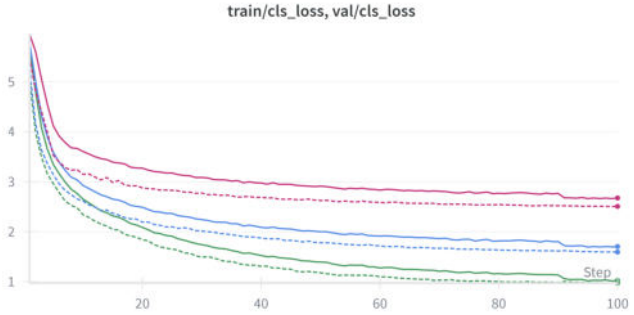

Fig. 18: Classification loss

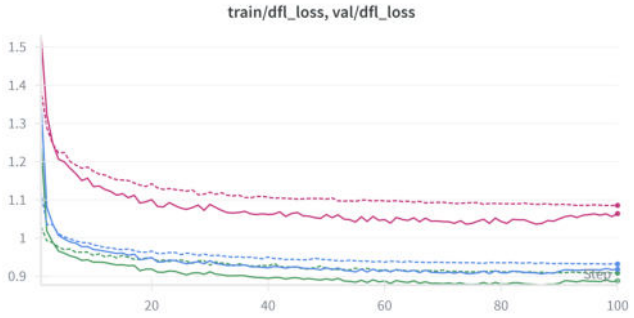

Fig. 19: Distribution focal loss

III.G Confidence behaviour

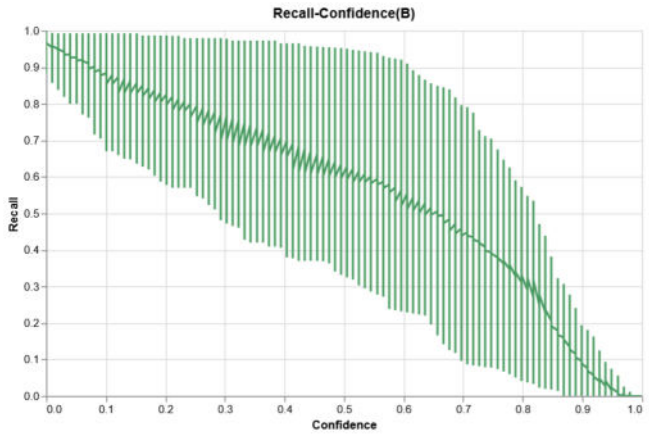

Fig. 20: Recall across different confidence thresholds, showing class-level spread

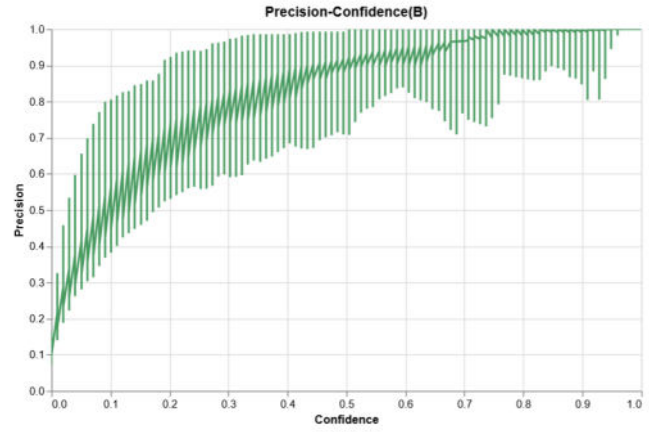

Fig. 21: Precision across different confidence thresholds, showing class-level spread.

III.H Qualitative Analysis

Fig. 22: Entire Model example detections

Fig. 23: Neck + Head example detections

Fig. 24: Head-only example detections

IV.A Higher model capacity leads to better performance

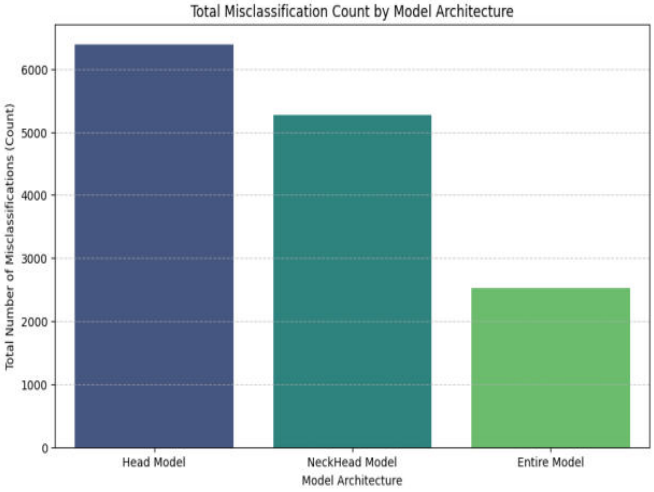

Fig. 25: Misclassification counts by architecture

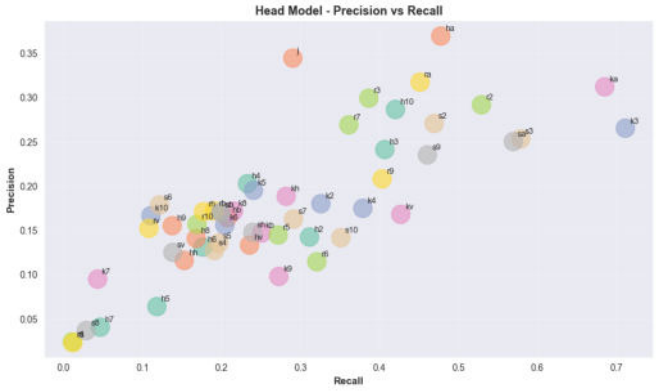

Fig. 26: Precision-Recall Plot - Head only model

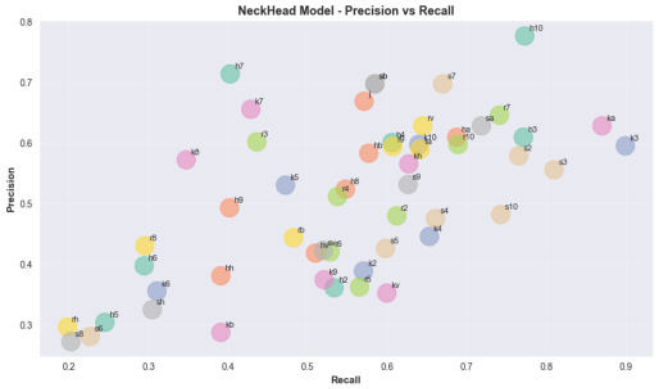

Fig. 27: Precision-Recall Plot - NeckHead model

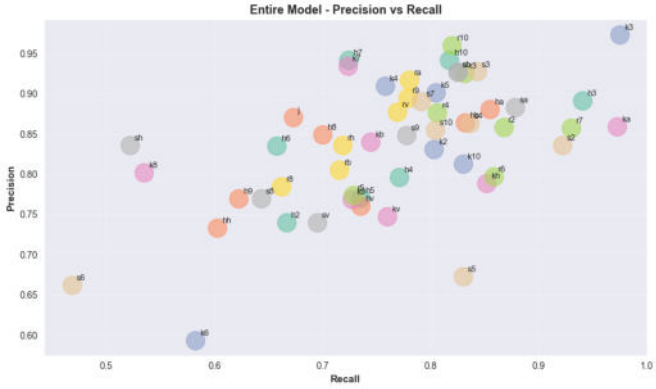

Fig. 28: Precision-Recall Plot - Entire model

IV.B Visually Similar Cards Are Misclassified More Often

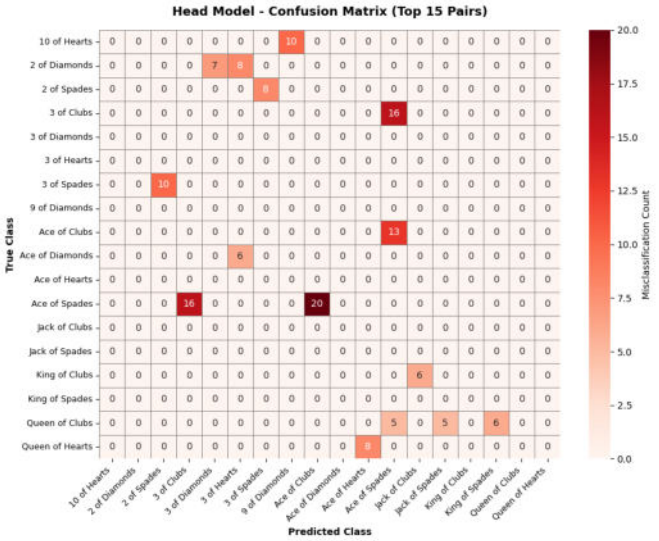

Fig. 29: Misclassifications Confusion Matrix for Head Only Model

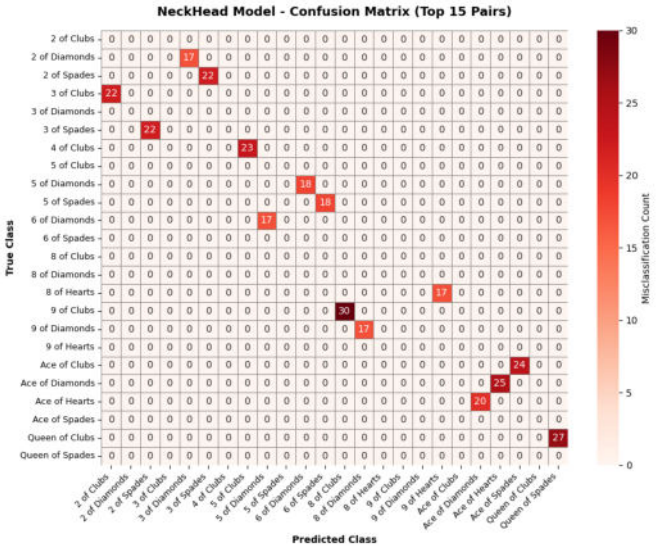

Fig. 30: Misclassifications Confusion Matrix for NeckHead Model